

Module 12: RESTful API Development

Module Overview

- Server-side HTML rendering executes presentation logic on the server, while client-side rendering runs it on the user's browser.
- Client-side applications can access server-stored data through either dedicated RESTful API endpoints or direct cloud service API calls.

- Two approaches to accessing a server-side database:
 - API endpoints (RESTful web services)
 - Direct cloud service API methods

- RESTful API endpoints are preferred over direct cloud API calls due to better customisation and business logic control.
- A single set of RESTful API endpoints can support multiple frontend interfaces, improving scalability and long-term maintainability.

- API endpoints are preferred because they:
 - Enable customisable business logic tailored to specific needs
 - Provide scalability and maintainability for multiple frontend applications

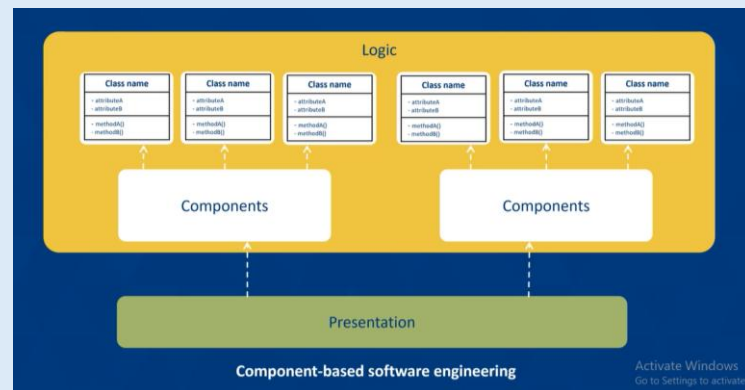
- This module introduces RESTful web services and walks through implementation using both JavaScript and Python.

Overview of Service-Oriented Architecture (SOA) and Microservices Architecture

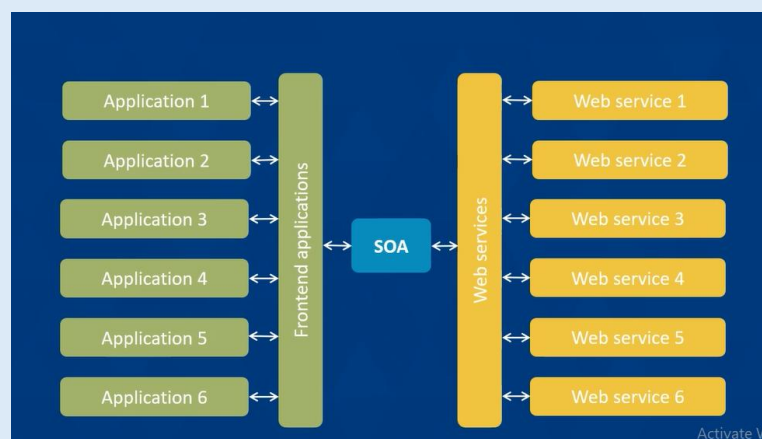
- Object-oriented programming promotes modularity and reuse but results in complex backend structures that complicate frontend integration.

- Structures large-scale software using classes and objects
- Promotes modularity, reusability, and maintainability
- Generates numerous specific and detailed classes in the backend
- Limits class behaviours, making integration challenging
- Increases complexity in frontend-backend coupling
- Complicates maintenance due to backend changes affecting the frontend

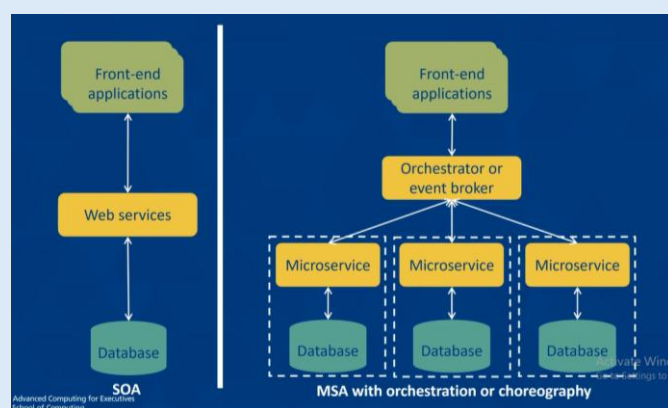
- Component-based architecture simplifies integration by grouping related classes into standalone components, reducing direct frontend-backend interactions.



- Despite its benefits, component-based architecture lacks flexibility in diverse environments due to its reliance on native method calls.
- A monolithic SOA setup is illustrated, where backend web services interact with various frontend applications through standard HTTP protocols.

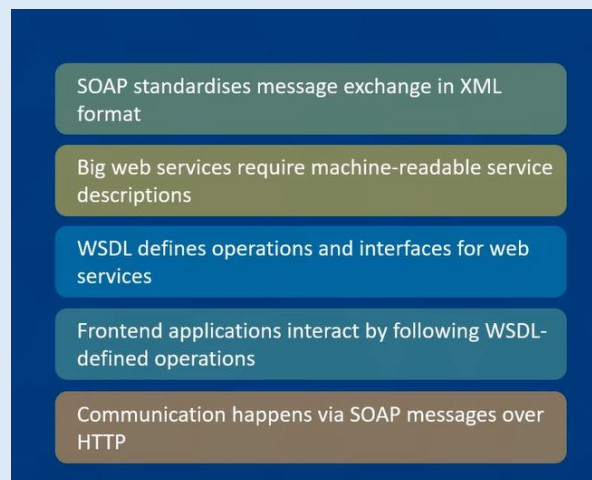


- Microservices architecture replaces the monolithic SOA model by enabling independently deployed web services that support flexible and parallel development.

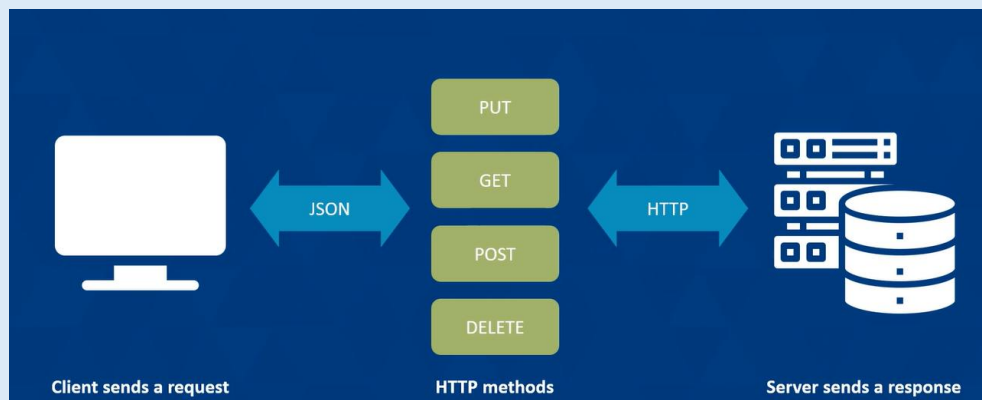


Introduction to RESTful Web Services

- Big web services use SOAP and XML with WSDL-defined operations, while RESTful services use HTTP methods with XML or JSON.



- RESTful web services simplify integration by removing the need for protocols like SOAP and interface definitions like WSDL.
- RESTful APIs follow standard HTTP methods—PUT, GET, POST, and DELETE—corresponding to the four CRUD operations.



- Compared to big web services, RESTful services are lighter, more scalable, loosely coupled, and better suited for mobile and IoT.

Lightweight, loosely coupled, and scalable architecture

Simpler backend development for web applications

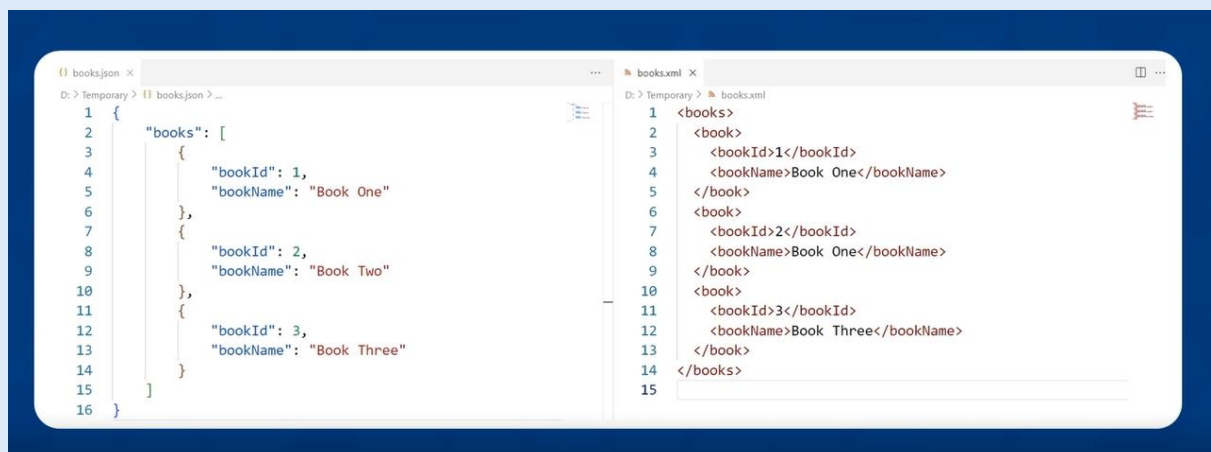
Ideal for mobile and IoT applications on low-power devices

- RESTful services exchange stateless, self-descriptive messages in plain text, using URLs to identify endpoints and resources.

Self-descriptive messages:
Messages are plain text (XML/JSON) and do not rely on complex schemas like SOAP

Introduction to JSON

- JSON and XML are both plain text formats used in RESTful web services, but JSON is generally more widely preferred.



- Both formats are human readable, hierarchical, and compatible with modern web browsers through the XMLHttpRequest object.

JSON and XML are widely used and supported

XMLHttpRequest enables fetching data in modern web browsers

- JSON is more concise than XML, omitting end tags and angle brackets, which makes it faster and easier to read and write.
- JSON integrates well with JavaScript and Python due to structural similarities with JavaScript objects and Python dictionaries.

The screenshot shows two code editors side-by-side, illustrating JSON handling in JavaScript and Python. The left editor, titled 'jsdemo.js', shows JavaScript code using `JSON.parse()` and `JSON.stringify()` to convert between JSON strings and JavaScript objects. The right editor, titled 'jsdemo.py', shows Python code using `json.loads()` and `json.dumps()` for the same purpose. Both editors have a terminal window at the bottom showing the output of the code execution.

JavaScript – JSON to/from JavaScript object

```
1 var jsonStr = '{ "books": [{"bookId": 1,"bookName": "Book One"}] }';
2 var obj = JSON.parse(jsonStr);
3
4 console.log(typeof(obj));
5 console.log(obj);
6
7 jsonStr = JSON.stringify(obj)
8
9 console.log(typeof(jsonStr));
10 console.log(jsonStr);
11
```

Python – JSON to/from Python dict

```
1 import json
2
3 jsonStr = '{ "books": [{"bookId": 1,"bookName": "Book One"}] }'
4 obj = json.loads(jsonStr)
5
6 print(type(obj))
7 print(obj)
8
9 jsonStr = json.dumps(obj)
10
11 print(type(jsonStr))
12 print(jsonStr)
13
```

Terminal output for JavaScript:

```
D:\Temporary> node jsdemo.js
object
{
  books: [
    { bookId: 1, bookName: 'Book One' },
    { bookId: 2, bookName: 'Book Two' },
    { bookId: 3, bookName: 'Book Three' }
  ]
}
string
{"books":[{"bookId":1,"bookName":"Book One"}, {"bookId":2,"bookName":"Book Two"}, {"bookId":3,"bookName":"Book Three"}]}
```

Terminal output for Python:

```
d:\Temporary> python jsdemo.py
<class 'dict'>
{'books': [{'bookId': 1, 'bookName': 'Book One'}, {'bookId': 2, 'bookName': 'Book Two'}, {'bookId': 3, 'bookName': 'Book Three'}]}
<class 'str'>
{"books": [{"bookId": 1, "bookName": "Book One"}, {"bookId": 2, "bookName": "Book Two"}, {"bookId": 3, "bookName": "Book Three"}]}
```

- JSON is commonly used for both input and output data in RESTful operations such as GET, POST, and PUT requests.

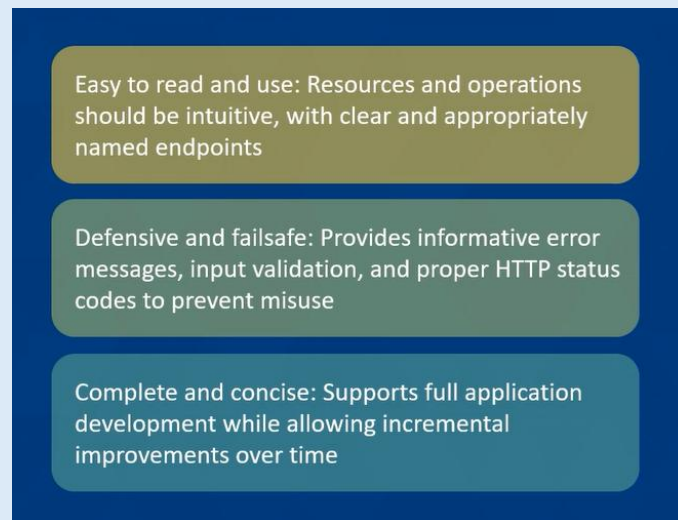
Easy to use for both input and output data

GET operation returns data as a JSON string

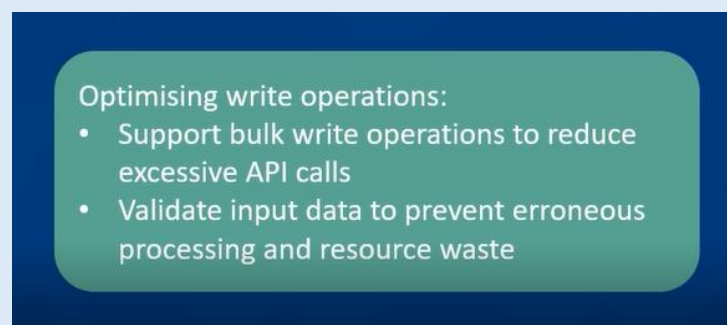
PUT and POST operations accept JSON input

Best Practices in RESTful API Design

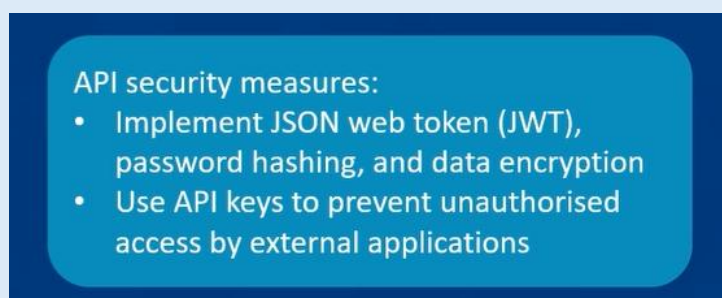
- A well-designed API should be intuitive, defensive in handling errors, and complete enough to support full application development.



- Informative feedback, meaningful HTTP status codes, and clear input validation enhance usability and prevent incorrect API usage.
- Efficient read operations should avoid retrieving all records, favour pagination, and limit unnecessary relationship data fetches.
- Bulk write operations and thorough data validation improve performance and reduce the number of API calls during transactions.

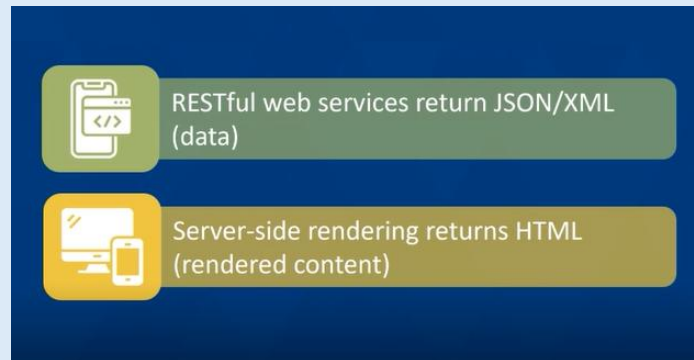


- Robust security practices like JSON web tokens, encryption, and application keys are essential to protect API endpoints from misuse.

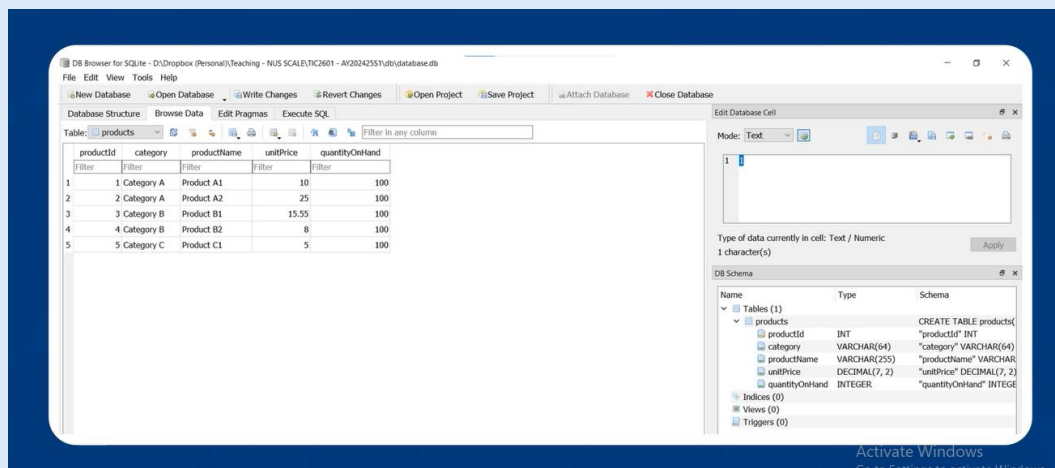


Creating RESTful API Endpoints with Expressjs

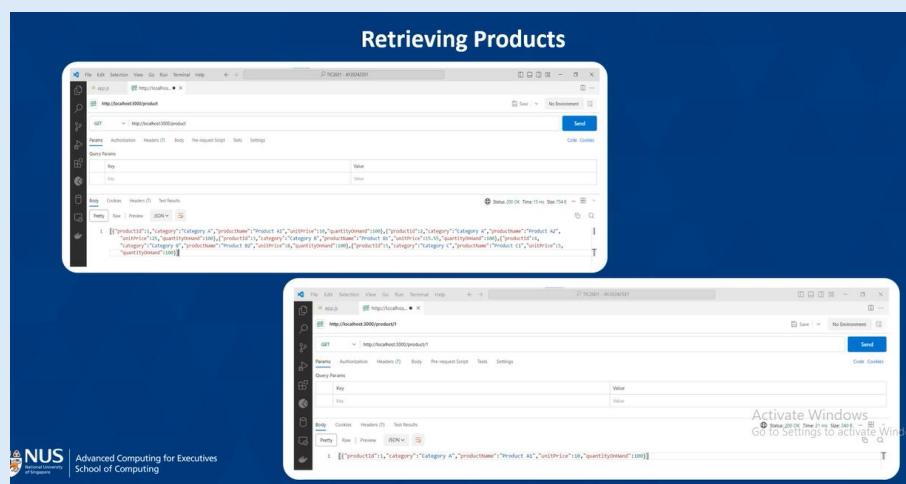
- RESTful web services using Express.js return data in JSON format, unlike server-side web apps that return HTML content.



- All data used in the RESTful web services is assumed to be stored in a database table of product records accessed using standard techniques.



- GET endpoints can retrieve all product records or a specific product using a route parameter, with proper error handling included.



- PUT and POST methods accept JSON data in the request body to create or update records, with checks for record existence.

```

1  const express = require('express');
2  const app = express();
3  const port = 3000;
4
5  app.use(express.json())
6
7  const sqlite3 = require('sqlite3').verbose()
8  const db = new sqlite3.Database('./db/database.db')
9
10 app.put('/product', (req, res) => {
11
12   const sql = `INSERT INTO products VALUES (${req.body.productId}, '${req.body.category}', '${req.body.productName}', ${req.body.unitPrice}, ${req.body.quantityOnHand})`;
13
14   db.prepare(sql).run([], function (err) {
15     if (err) {
16       res.status(400).send('Bad Request: productId already exists');
17     } else {
18       res.sendStatus(201);
19     }
20   });
21 })
22
23
24 app.listen(port, function () {
25   console.log('Express app listening on port ${port}!');
26 });

```

- DELETE operations use query string parameters to identify records and return appropriate status codes based on success or failure.

Pass productId as Query Parameter & Validate

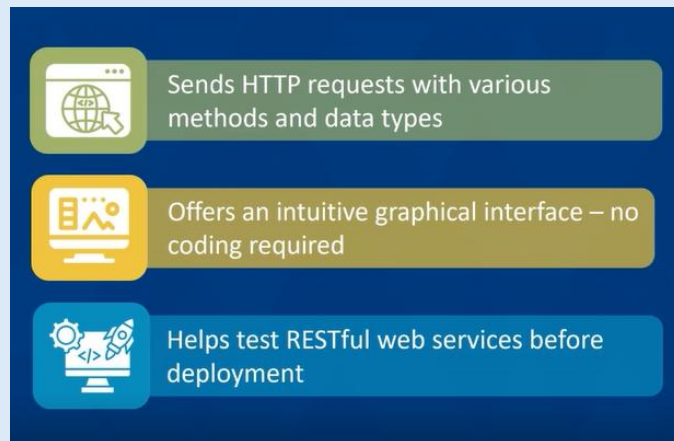
```

1  const express = require('express');
2  const app = express();
3  const port = 3000;
4
5  app.use(express.json())
6
7  const sqlite3 = require('sqlite3').verbose()
8  const db = new sqlite3.Database('./db/database.db')
9
10 app.delete('/product', (req, res) => {
11
12   const sql = `DELETE FROM products WHERE productId = '${req.query.productId}'`;
13
14   db.prepare(sql).run([], function (err) {
15     if (err) {
16       res.status(500).send('Unknown error');
17     } else {
18       if (this.changes == 1) {
19         res.sendStatus(200);
20       } else {
21         res.status(400).send('Bad Request: productId does not exists');
22       }
23     }
24   });
25 })
26
27 app.listen(port, function () {
28   console.log('Express app listening on port ${port}!');
29 });

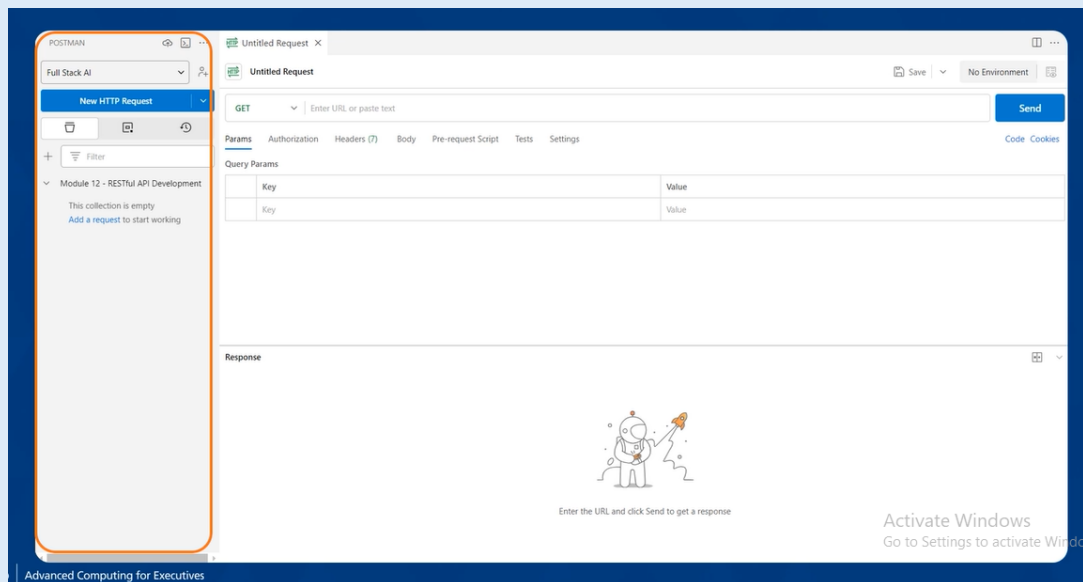
```

Testing RESTful API Endpoints with Postman

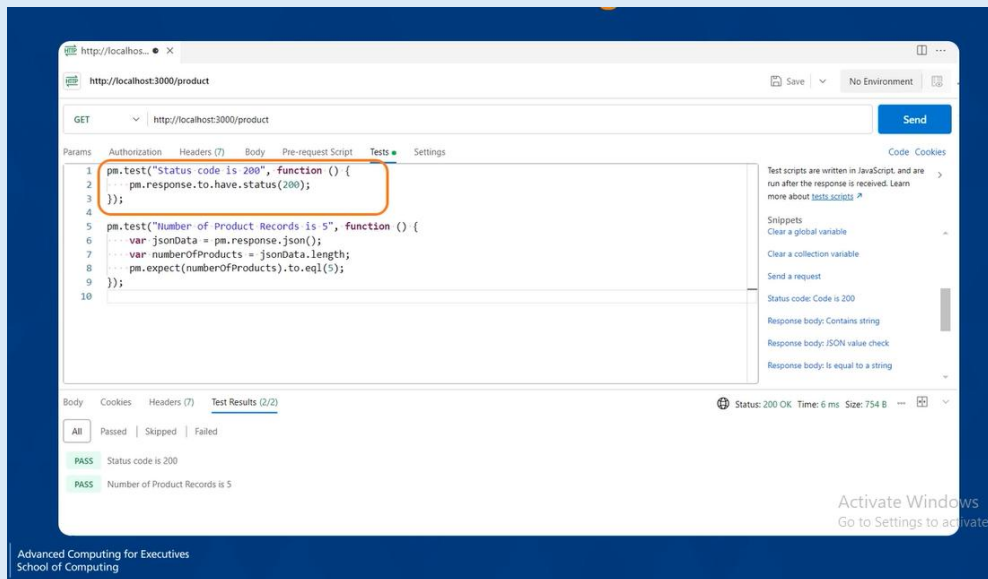
- Postman is a GUI-based tool that enables testing of RESTful web services by sending HTTP requests with various methods and inputs.



- The Visual Studio Code plugin version of Postman allows users to manage workspaces, organise requests in collections, and save configurations.



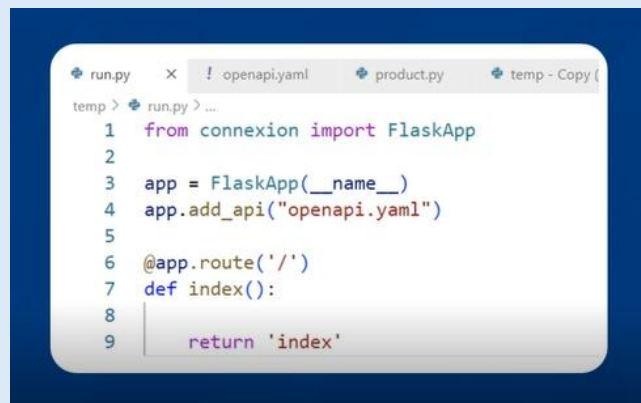
- HTTP requests in Postman can include URL, query strings, and request body, with results displayed in the response section after execution.
- Response data, such as JSON output and HTTP status codes like 200, help verify correct behaviour of RESTful endpoints during testing.



- Automated test scripts in Postman validate responses and status codes, improving reliability and streamlining backend development workflows.

Creating RESTful API Endpoints with Flask and Connexion

- Connexion, built on Flask, enables development of RESTful web services in Python using a declarative OpenAPI or Swagger specification.



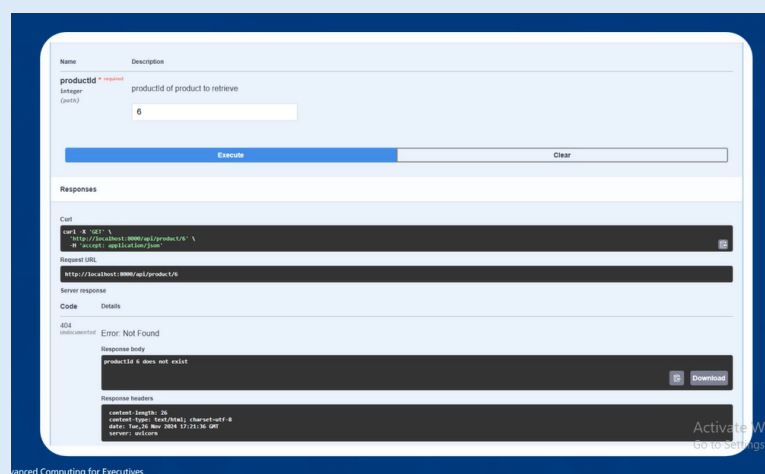
- A single Flask application can serve both HTML and JSON data, allowing integration of server-rendered pages with RESTful endpoints.
- The OpenAPI YAML file defines endpoints and HTTP methods, mapping each one to specific Python functions through operationId declarations.



- Python functions use SQLite to query product data, convert tuples to dictionaries, and return JSON responses along with meaningful HTTP status codes.

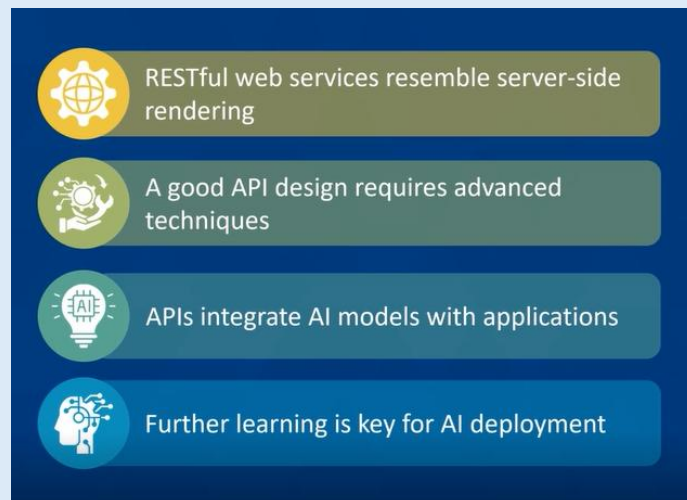


- Swagger UI auto-generates interactive documentation that allows direct testing of RESTful endpoints, replacing the need for Postman during early validation.



Module Summary

- RESTful web services resemble server-side rendering but focus on returning data instead of HTML content.
- Effective API design requires advanced techniques, including robust error handling, validation, and security.



- Effective RESTful services require attention to error handling, security, performance, and scalability.
- Express.js, Connexion, and Flask are essential tools for building and experimenting with RESTful services.
- RESTful APIs will play a key role in upcoming modules on AI model deployment and integration.